

# COMP 3069

# Computer Graphics



Lecture 2:  
Rendering

Autumn 2025

# Contents

- ◆ Define terminology: pixel and frame buffer etc.
- ◆ Briefly describe 2D rendering and its stages
- ◆ Describe Vertex Specification and Triangle Assembly
- ◆ Explain the process of rasterisation
- ◆ Explain the process of calculating Barycentric coordinates, and 'inside test'
- ◆ Define a shader with the example of a simple Fragment shader



# 2D Rendering

Rendering (渲染) 是计算机将图形绘制到屏幕上的过程。它是从几何数据（如点、线、三角形）到最终彩色图像的转化过程。

- ◆ Rendering is the process of drawing images on the computer display
- ◆ The graphics programmer specifies vertices, which make up some triangles to be drawn
- ◆ The API assembles triangles from the vertices
- ◆ The API rasterises the triangles to calculate which pixels are inside each triangle
- ◆ The pixels inside triangles, called fragments, are shaded to calculate the colour
- ◆ The colours are displayed at the appropriate pixels

渲染 (Rendering) 就是把数学描述的几何对象（如三角形）转换成屏幕上彩色像素的过程。

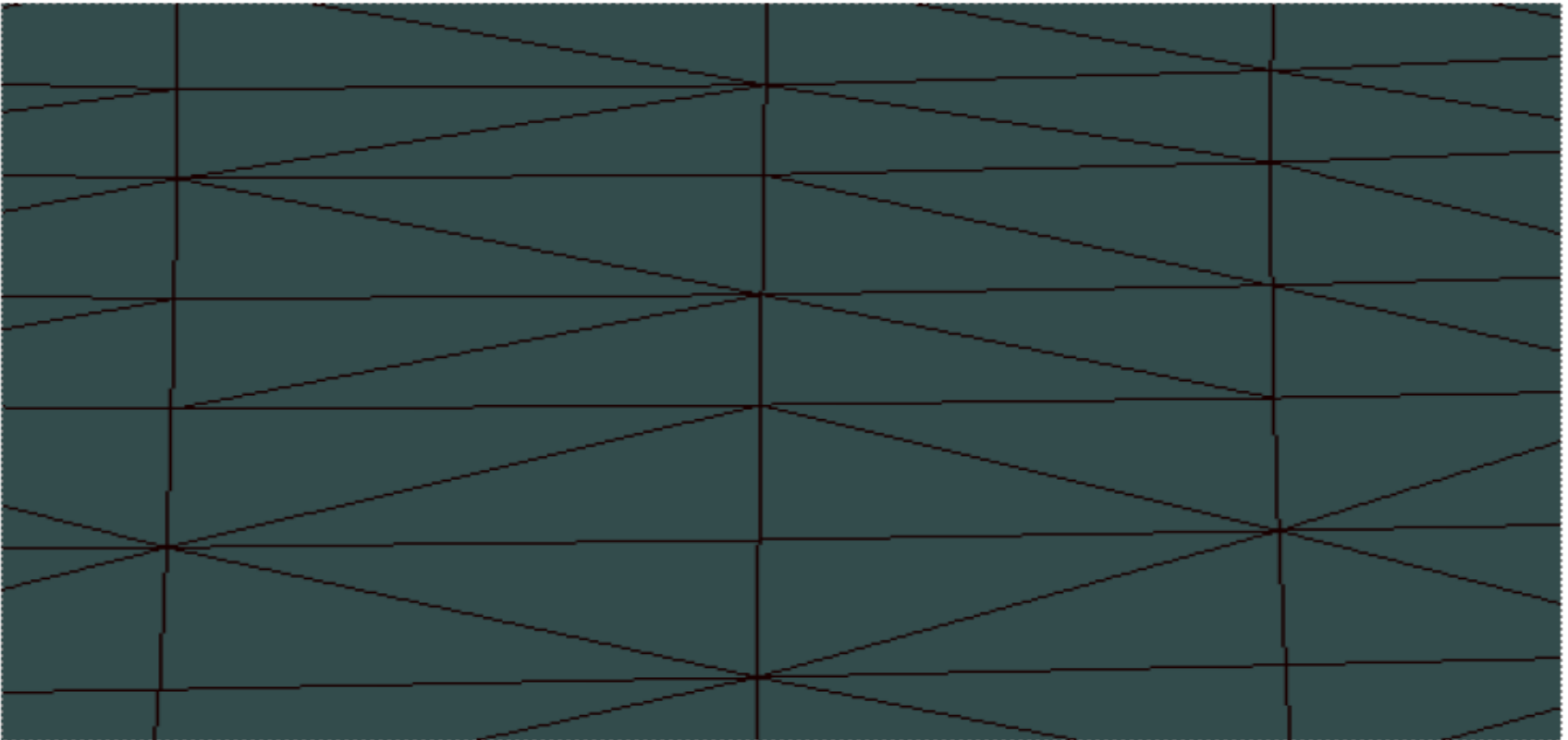
整个流程由图形 API（如 OpenGL）和 GPU 共同完成。

| 阶段      | 说明          | 对应操作            |
|---------|-------------|-----------------|
| 1 顶点定义  | 定义几何形状的顶点   | 程序员提供           |
| 2 三角形组装 | 将顶点组合成几何图元  | API 组装          |
| 3 光栅化   | 确定哪些像素属于三角形 | GPU 自动完成        |
| 4 片段着色  | 计算每个像素的颜色   | Fragment Shader |
| 5 显示输出  | 将颜色显示在屏幕上   | Frame Buffer 输出 |



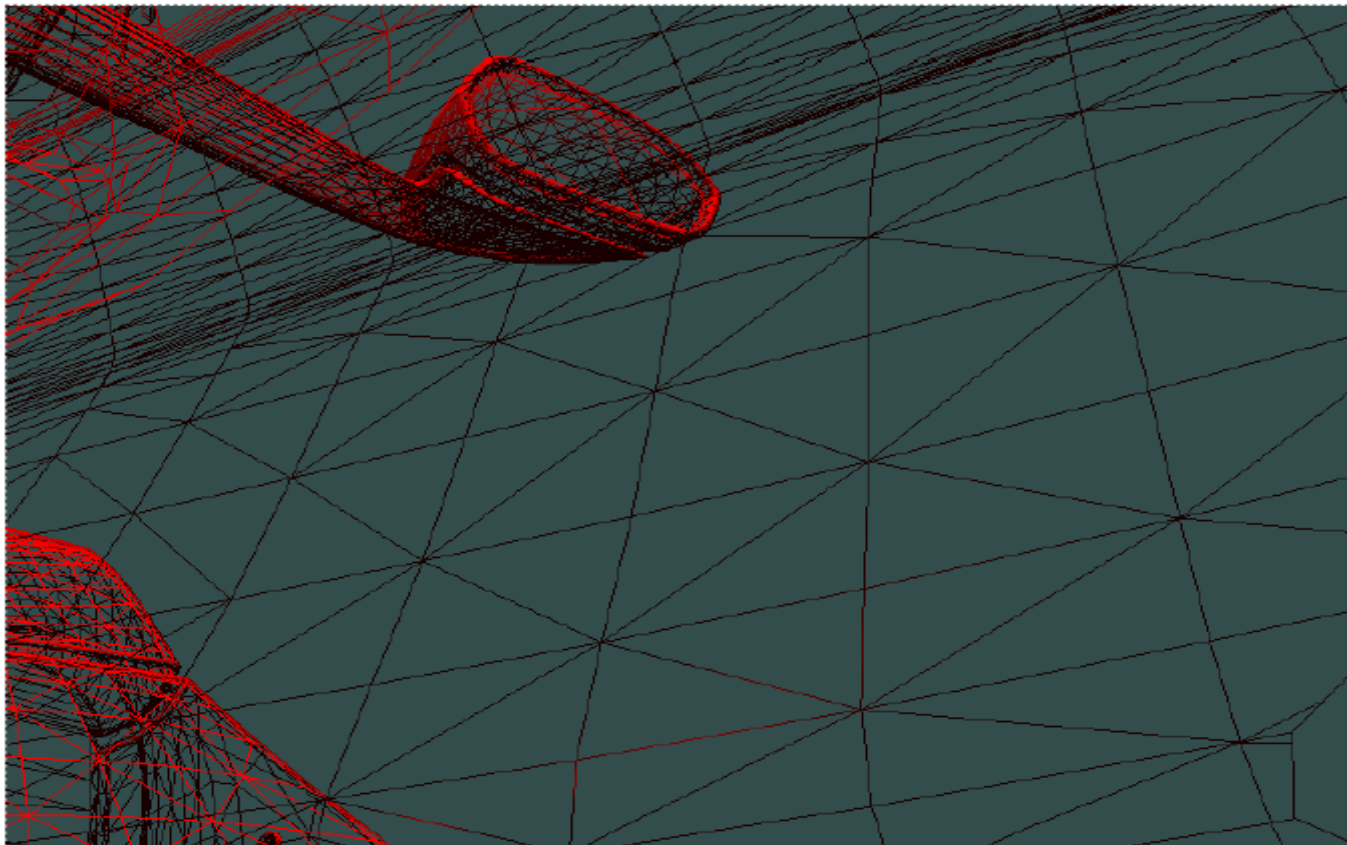
# Simple Triangles

- ◆ Images made up of triangles.



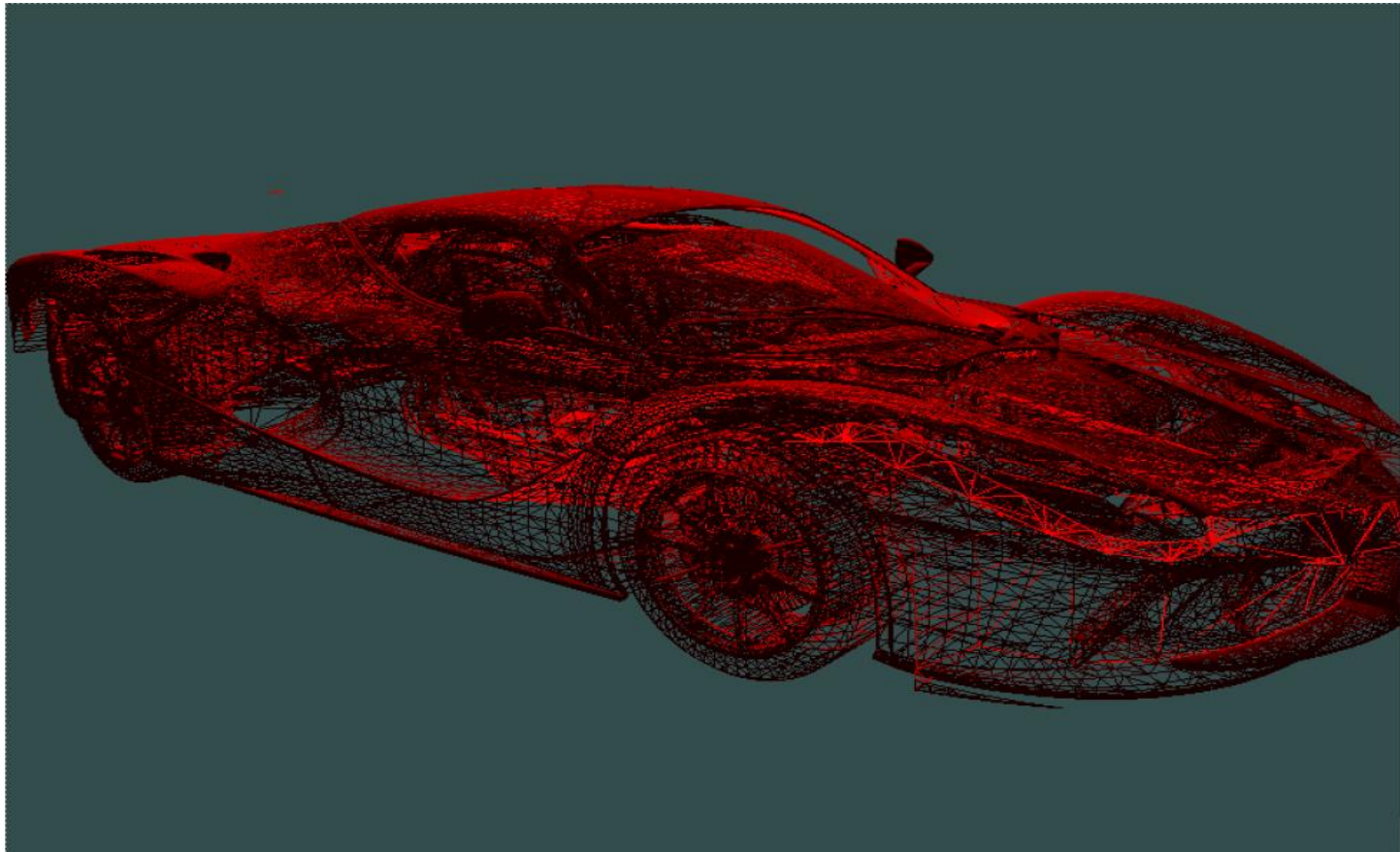
# More Triangles

- ◆ The more triangles are used, the more rich and detailed the image becomes



# Millions of Triangles

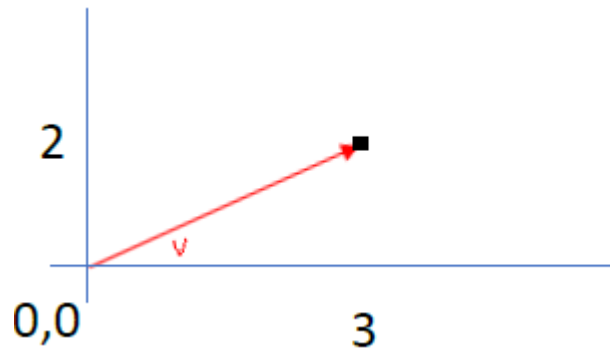
- ◆ Complex images are made up of millions of triangles



# Vertex

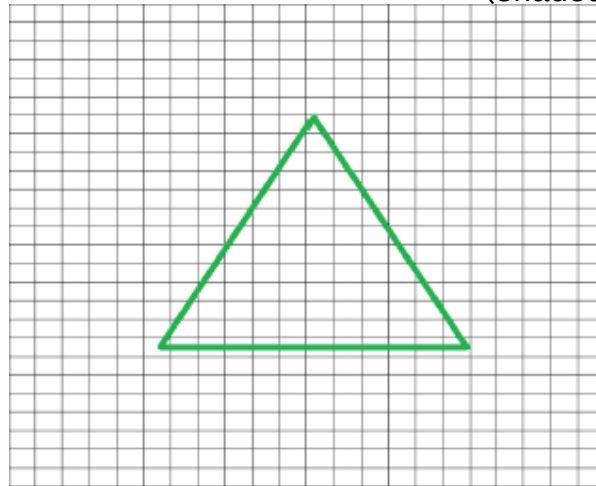
- ◆ A vertex is a point in space and is used to model geometry
- ◆ A vertex can be presented using a vector, which is like an arrow. A vector can be specified as  $v = (3, 2, 0)$  or  $v = (3, 2, 0)^T$
- ◆ A vector can be used to represent a vertex position by placing the start of the vector at the origin of the coordinate space, and the end at the vertex position

向量从坐标原点出发，  
指向顶点所在位置。



# Triangle

- ◆ A triangle is made of 3 vertices
- ◆ A raster display is a common form of computer display, made up of pixels 光栅显示器 (Raster Display) 是由许多像素 (pixels) 组成的网格。
- ◆ A pixel is a picture element
- ◆ A **fragment** is a piece of a square and is drawn to a pixel 在三角形内部的像素称为 片段 (fragment)。
- ◆ In an image, all of the squares inside triangle can be visualised as fragments 渲染时, 三角形内部的所有像素 (即片段) 都会被着色 (shaded) 。



# Colour Buffer and Shader

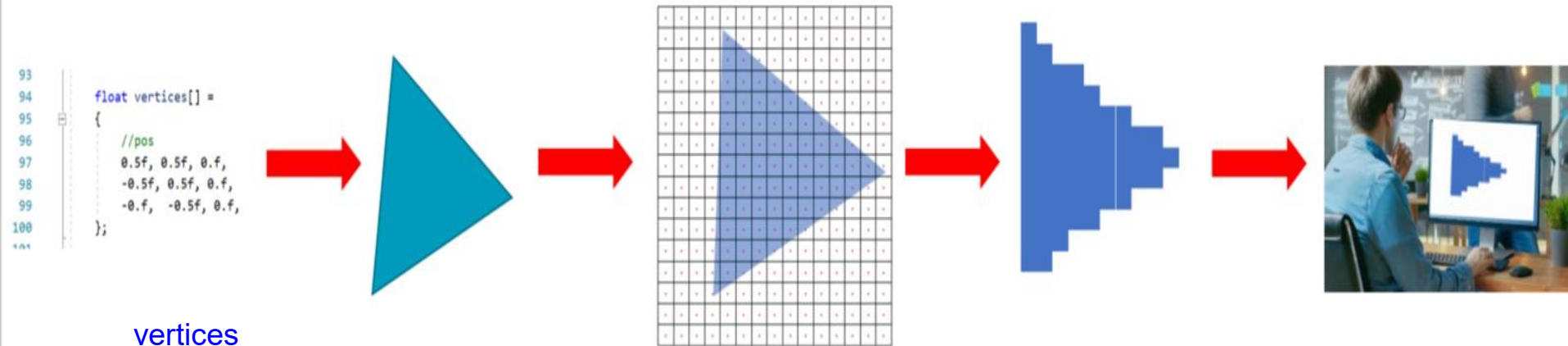
- ◆ A section of memory called a frame buffer (or colour buffer) stores the colour values that will be used at each pixel
- ◆ A shader is a computer program that calculates the appropriate levels of light, darkness, colour, etc.
- ◆ Graphics programming involves writing shaders
- ◆ In this course, we will focus on writing Vertex Shaders and Fragment Shaders

■ 内容解释:

- 颜色缓冲区 (Colour Buffer)
  - 是显存的一部分, 用来存储每个像素的颜色信息。
- 着色器 (Shader)
  - 是 GPU 上运行的小程序, 用于计算像素的最终颜色、光照、阴影、反射等。
- 类型包括:
  - Vertex Shader (顶点着色器)
  - Fragment Shader (片段着色器)



# What is Rendering?



vertices

|      |
|------|
| 0.5  |
| 0.5  |
| 0    |
| -0.5 |
| 0.5  |
| 0    |
| -0   |
| -0.5 |
| 0    |

渲染 = 顶点数据 → 三角形 → 像素 (片段) → 屏幕图像。



The University of  
Nottingham

UNITED KINGDOM • CHINA • MALAYSIA

# Rendering Stages

- ◆ **Vertex Specification**      Vertex Specification (顶点定义) : 程序员提供顶点坐标。
  - The graphics programmer specifies the locations of vertices
- ◆ **Vertex Shader**      Vertex Shader (顶点着色器) : GPU 处理每个顶点 (例如平移、旋转、投影)。
  - Vertex processing by the vertex shader moves the vertices around
- ◆ **Rasterisation**      Rasterisation (光栅化) : 确定哪些像素在三角形内部。
  - Rasterisation is the process of figuring out which pixels are inside each of the triangles
- ◆ **Fragment Shader**      Fragment Shader (片段着色器) : 计算每个像素 (片段) 的颜色值。
  - The colour of a fragment is calculated by the fragment shader



# Vertex Specification

- ◆ The programmer creates an array of vertices to be rendered.
- ◆ A simple example of vertex specification is

```
float vertices[] =
```

```
{
```

```
    //x
```

```
    //y
```

```
    //z
```

```
    -1.f,
```

```
    0.f,
```

```
    0.f,
```

```
    //v0
```

```
    0.f,
```

```
    -1.f,
```

```
    0.f,
```

```
    //v1
```

```
    -1.f,
```

```
    -1.f,
```

```
    0.f,
```

```
    //v2
```

```
}
```

vertices

-1

0

0

0

-1

0

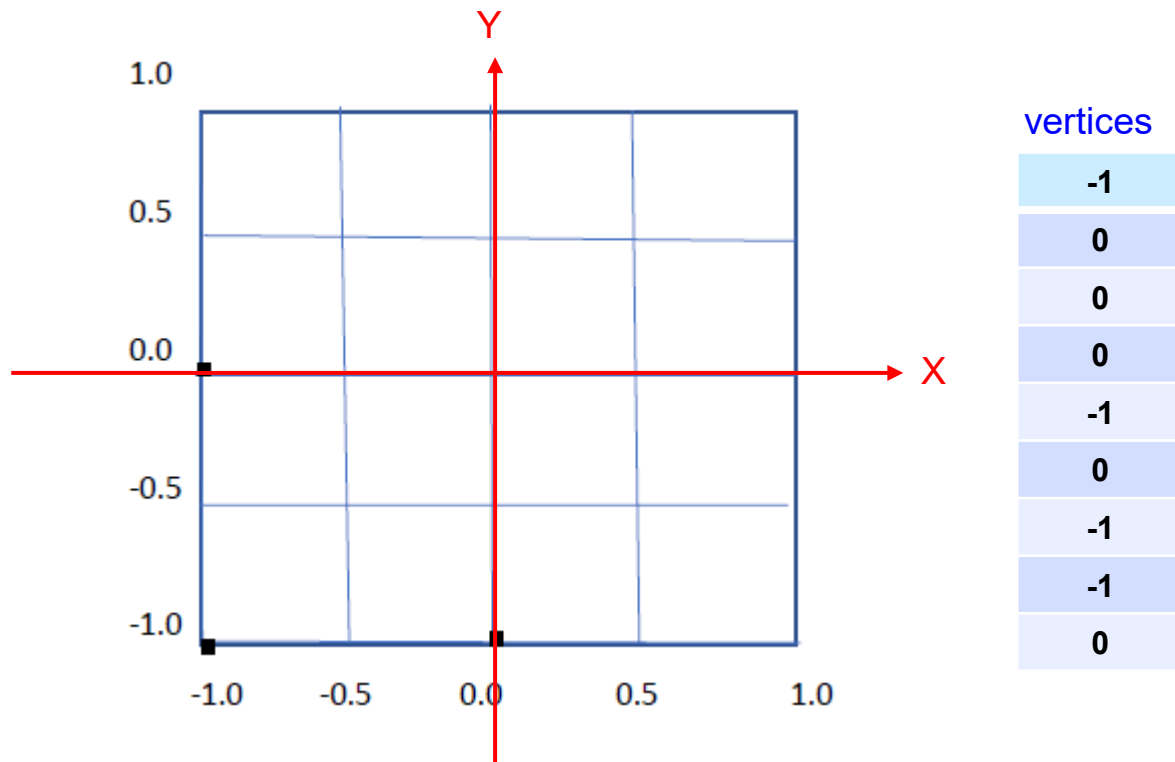
-1

-1

0

# Position of Vertex

- ◆ The position of each vertex is shown on a graph below.
- ◆ Since we are working in 2D, there is no depth to the figure and each vertex has a z value of 0.



# Triangle Specification

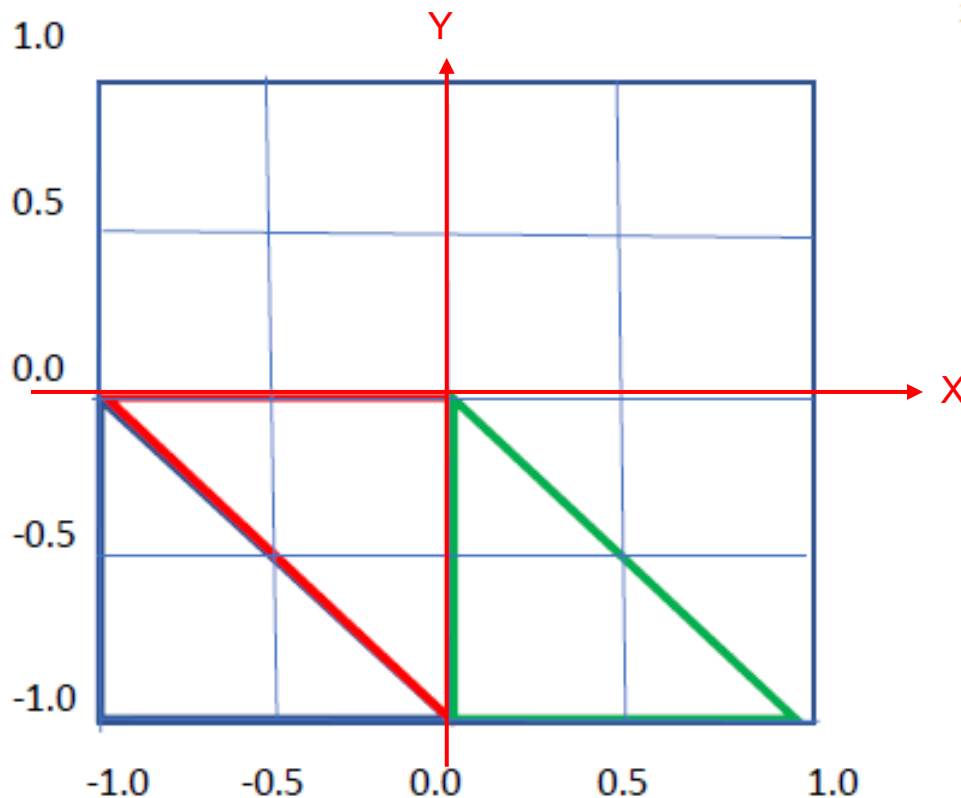
- ◆ A triangle is 3 vertices.
- ◆ It is the simplest 2-Dimensional shape, i.e. with the fewest vertices.
- ◆ Here is a triangle specification

```
float vertices[] =  
{  
  
    //blue triangle  
    -1.f, 0.f, 0.f,  
    0.f, -1.f, 0.f,  
    -1.f, -1.f, 0.f,  
  
    //red triangle  
    -1.f, 0.f, 0.f,  
    0.f, 0.f, 0.f,  
    0.f, -1.f, 0.f,  
  
    //green triangle  
    0.f, 0.f, 0.f,  
    1.f, -1.f, 0.f,  
    0.f, -1.f, 0.f,  
  
}
```



# Triangle Assembly

- ◆ The blue triangle is assembled first, then the red triangle, and lastly the green triangle.



```
float vertices[] =  
{  
    //blue triangle  
    -1.f, 0.f, 0.f,  
    0.f, -1.f, 0.f,  
    -1.f, -1.f, 0.f,  
  
    //red triangle  
    -1.f, 0.f, 0.f,  
    0.f, 0.f, 0.f,  
    0.f, -1.f, 0.f,  
  
    //green triangle  
    0.f, 0.f, 0.f,  
    1.f, -1.f, 0.f,  
    0.f, -1.f, 0.f,  
}
```

# Rasterisation and Shading

- ◆ The pseudocode for rasterisation and shading is as follows.

```
for each pixel y in Y dimension
{
    for each pixel x in X dimension
    {
        for each triangle t
        {
            if pixel x,y is inside triangle t
            {
                calculate the fragment colour
            }
        }
    }
}
```

Rasterisation

Shading

阶段

说明

Rasterisation (光栅化)

判断像素是否在三角形内部。

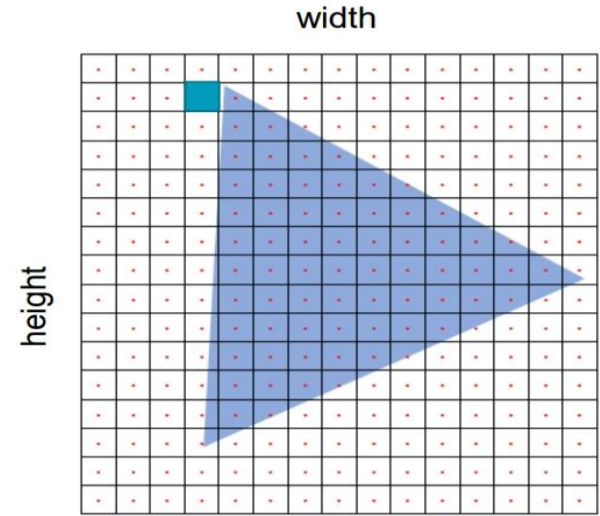
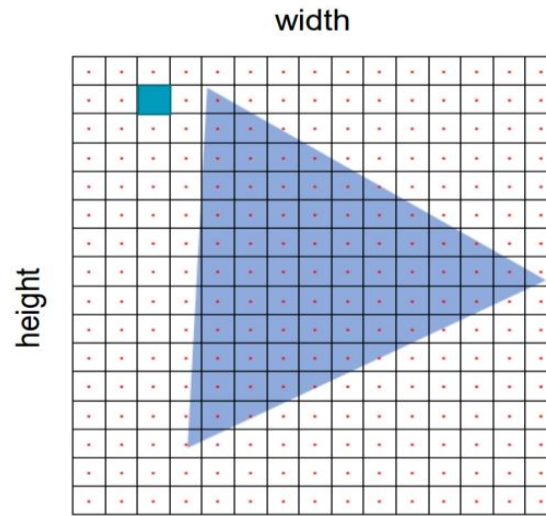
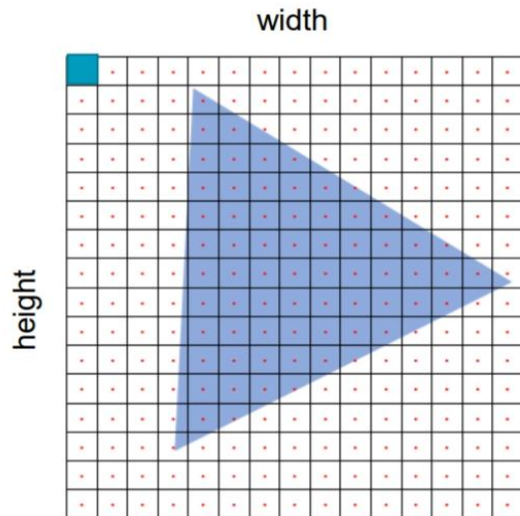
Shading (着色)

计算该像素的颜色 (考虑光线、纹理等)。

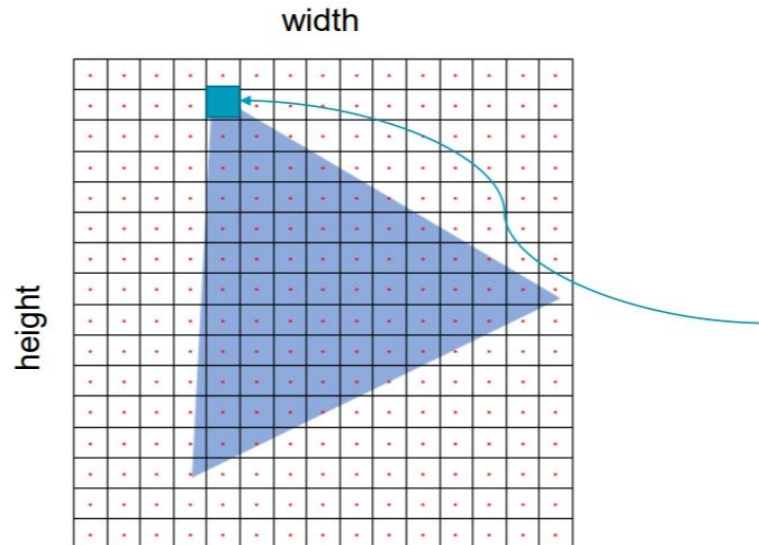


# Rasterisation and Shading

## Rasterisation



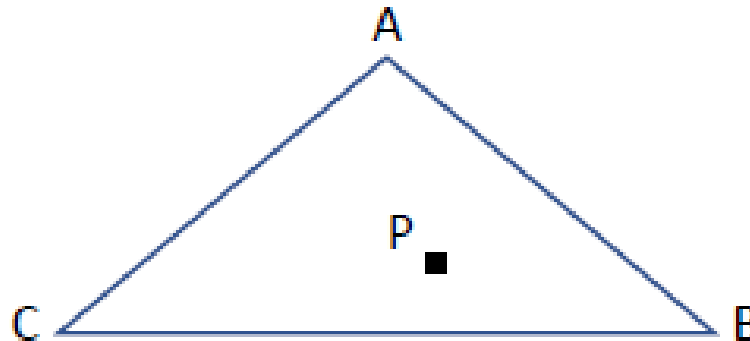
## Shading



Execute Fragment Shader  
for this fragment

# Barycentric Coordinates

- ◆ Barycentric coordinates are a way of calculating whether a point is inside a triangle, called the ‘inside test’.
- ◆ Suppose we have a triangle made of 3 vertices, A, B, and C, and a point P



- 重心坐标是一种数学方法，用来判断一个点 P 是否在三角形 ABC 内部。  
➡ 这也叫作 “inside test (内部测试)”。
- 对于任意三角形 ABC 和点 P，我们可以用三个数  $\alpha$ 、 $\beta$ 、 $\gamma$  来表示点 P 相对于三个顶点的“权重关系”。

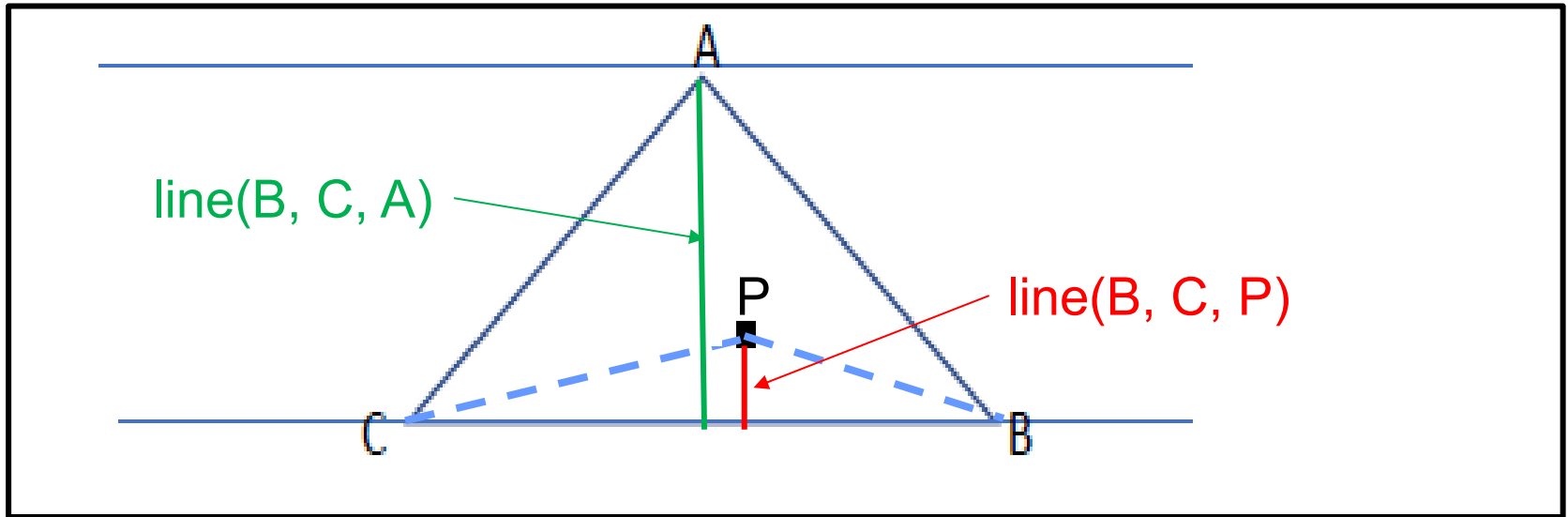
📊 核心思想：

- 如果  $\alpha$ 、 $\beta$ 、 $\gamma$  都在  $[0,1]$  范围内，且  $\alpha+\beta+\gamma=1$ ，则点 P 在三角形内部。



# Barycentric Coordinate $\alpha$

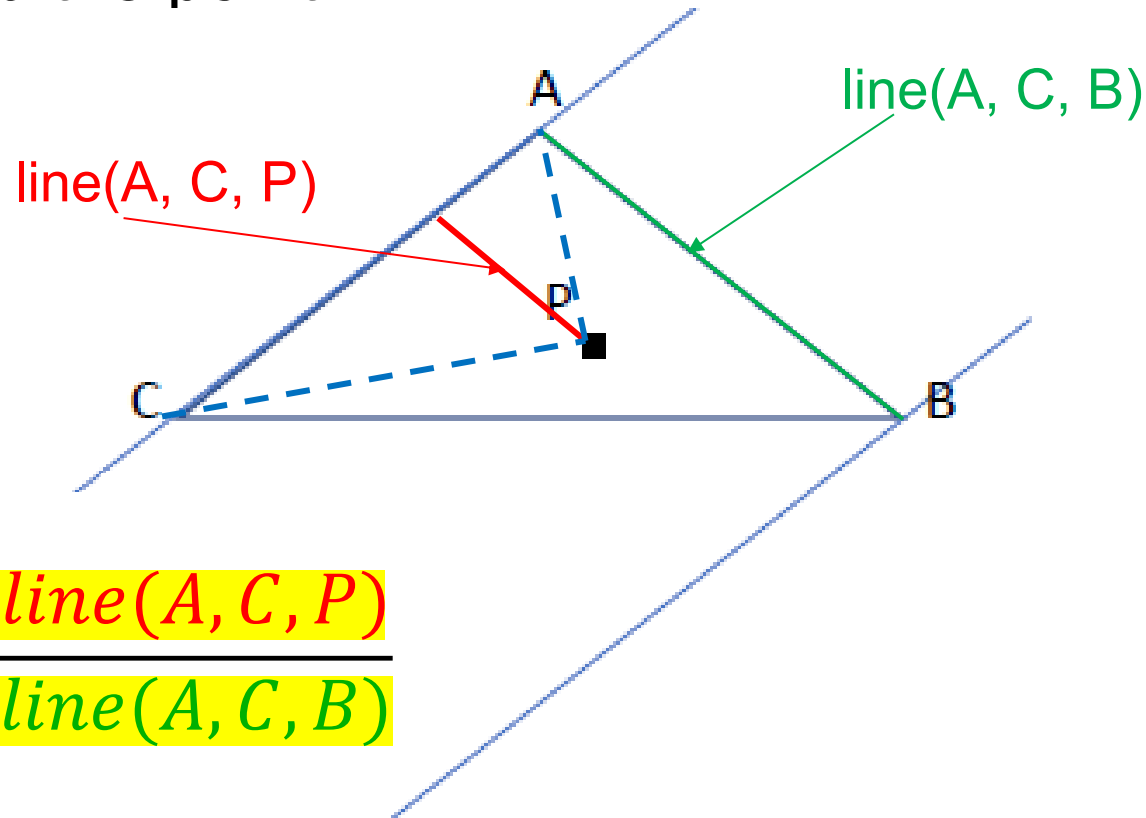
- ◆ The coordinate  $\alpha$  (alpha) corresponds to the normalised linear distance of P between the line BC and the point A



$$\alpha = \frac{line(B, C, P)}{line(B, C, A)}$$

# Barycentric Coordinate $\beta$

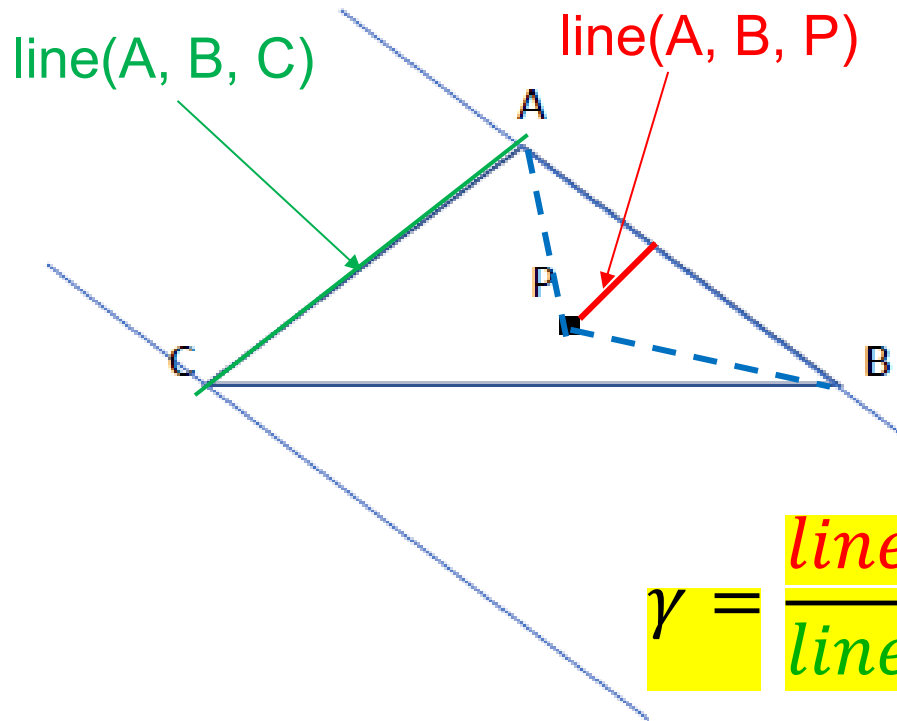
- ◆ The coordinate  $\beta$  (beta) corresponds to the normalised linear distance of P between the line AC and the point B



$$\beta = \frac{\text{line}(A, C, P)}{\text{line}(A, C, B)}$$

# Barycentric Coordinate $\gamma$

- ◆ The coordinate  $\gamma$  (gamma) corresponds to the normalised linear distance of P between the line AB and the point C



$$\gamma = \frac{\text{line}(A, B, P)}{\text{line}(A, B, C)}$$

# Calculation of $\alpha$ , $\beta$ and $\gamma$

- ◆ We can calculate  $\alpha$ ,  $\beta$  and  $\gamma$  as follows

$$\alpha = \frac{\text{line}(B, C, P)}{\text{line}(B, C, A)}$$

$$\beta = \frac{\text{line}(A, C, P)}{\text{line}(A, C, B)}$$

$$\gamma = \frac{\text{line}(A, B, P)}{\text{line}(A, B, C)}$$

- 如果  $\alpha$ 、 $\beta$ 、 $\gamma$  都在  $[0, 1]$  之间  $\rightarrow$  点  $P$  在三角形内部。
- 如果任意一个小于 0 或大于 1  $\rightarrow$  点  $P$  在外部。

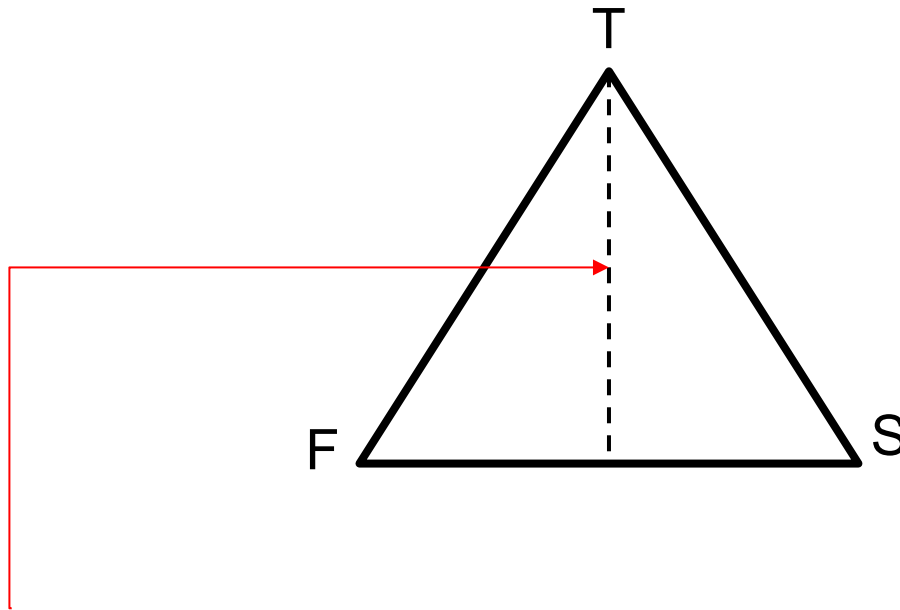
■ 补充：

- 当  $\text{line}(A, B, P) = 0 \rightarrow P$  在  $AB$  线上；
- 当  $\text{line}(B, C, P) = 0 \rightarrow P$  在  $BC$  线上。

- ◆ It is easy to see that iff  $\alpha$ ,  $\beta$  and  $\gamma$  are all in the range  $[0, 1]$ , then the point is inside the triangle.
- ◆  $\text{line}(A, B, P) = 0$  iff  $P$  is on line  $AB$

# Calculation of $\alpha$ , $\beta$ and $\gamma$

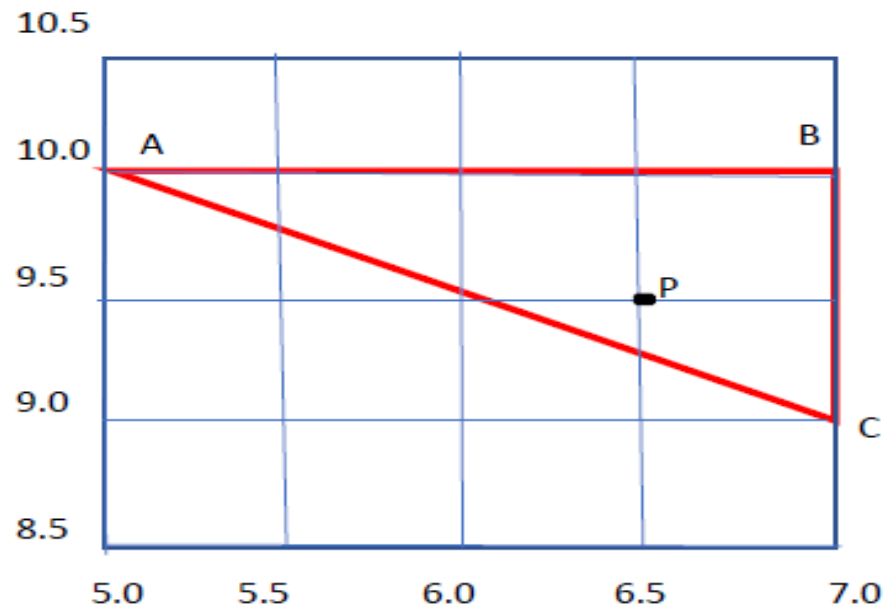
- Given a general triangle  $F$ ,  $S$ ,  $T$ , where  $F$  and  $S$  are the first and second base vertices respectively, and  $T$  is the top of the triangle.



$$\text{line}(F, S, T) = (S_y - F_y)T_x + (F_x - S_x)T_y + S_xF_y - F_xS_y$$

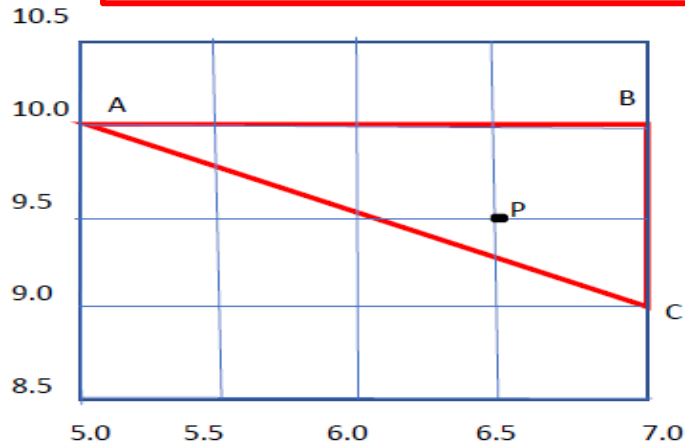
# Inside Test

- ◆ Compute the Barycentric Coordinates  $\alpha$ ,  $\beta$  and  $\gamma$  for the following triangle ABC, and point P. Is the point inside the triangle?



# Calculation of $\alpha$

$$\text{line}(F, S, T) = (S_y - F_y)T_x + (F_x - S_x)T_y + S_xF_y - F_xS_y$$



$$\alpha = \frac{\text{line}(B, C, P)}{\text{line}(B, C, A)}$$

$$\text{line}(B, C, P) = (9 - 10)6.5 + (7 - 7)9.5 + 7 * 10 - 7 * 9$$

$$\text{line}(B, C, P) = -6.5 + 0 + 70 - 63$$

$$\text{line}(B, C, P) = 0.5$$

$$\text{line}(B, C, A) = (9 - 10)5 + (7 - 7)10 + 7 * 10 - 7 * 9$$

$$\text{line}(B, C, A) = 2$$

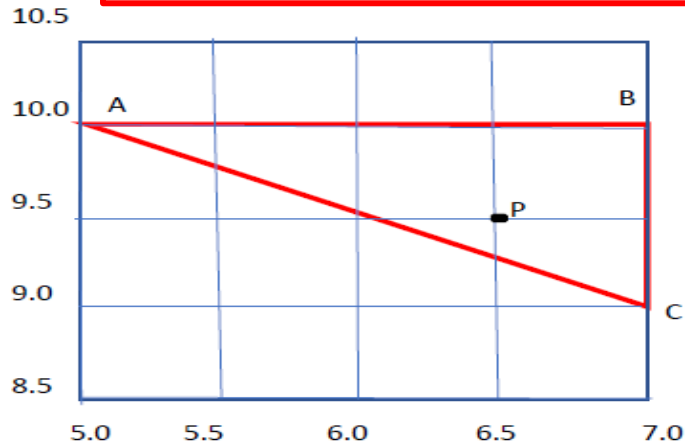
$$\alpha = \frac{0.5}{2}$$

$$\alpha = 0.25$$



# Calculation of $\beta$

$$\text{line}(F, S, T) = (S_y - F_y)T_x + (F_x - S_x)T_y + S_xF_y - F_xS_y$$



$$\beta = \frac{\text{line}(A, C, P)}{\text{line}(A, C, B)}$$

$$\text{line}(A, C, P) = (9 - 10)6.5 + (5 - 7)9.5 + 7 * 10 - 5 * 9$$

$$\text{line}(A, C, P) = -6.5 - 19 + 70 - 45$$

$$\text{line}(A, C, P) = -0.5$$

$$\text{line}(A, C, B) = (9 - 10)7 + (5 - 7)10 + 7 * 10 - 5 * 9$$

$$\text{line}(A, C, B) = -7 - 20 + 70 - 45$$

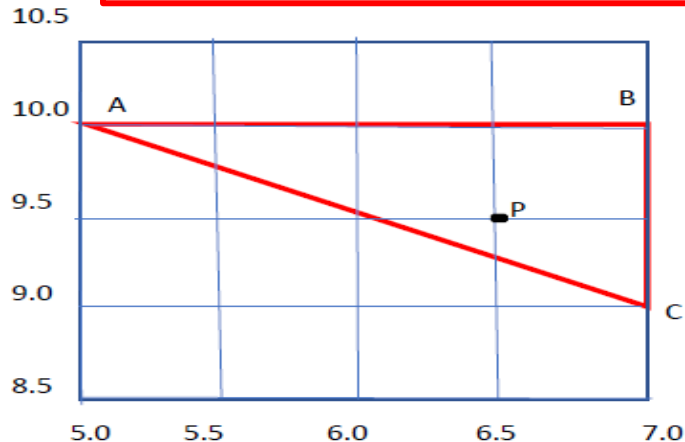
$$\text{line}(A, C, B) = -2$$

$$\beta = \frac{-0.5}{-2}$$

$$\beta = 0.25$$

# Calculation of $\gamma$

$$\text{line}(F, S, T) = (S_y - F_y)T_x + (F_x - S_x)T_y + S_x F_y - F_x S_y$$



$$\gamma = \frac{\text{line}(A, B, P)}{\text{line}(A, B, C)}$$

$$\text{line}(A, B, P) = (10 - 10)6.5 + (5 - 7)9.5 + 7 * 10 - 5 * 10$$

$$\text{line}(A, B, P) = 0 - 19 + 70 - 50$$

$$\text{line}(A, B, P) = 1$$

$$\text{line}(A, B, C) = (10 - 10)7 + (5 - 7)9 + 7 * 10 - 5 * 10$$

$$\text{line}(A, B, C) = 2$$

$$\gamma = \frac{1}{2}$$

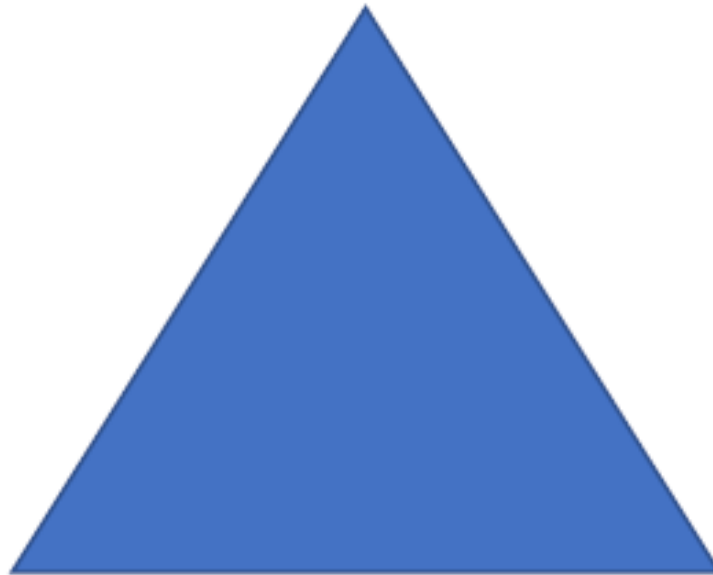
$$\gamma = 0.5$$

# Is the point $P$ inside the triangle?

$\alpha$ ,  $\beta$  and  $\gamma$  are all  $>0$  and  $<1$  so the point is algebraically inside the triangle.

# Fragment Shader

- ◆ A fragment shader calculates the colour of a fragment.
- ◆ In the following example image, the fragment shader will just return the colour blue for each fragment.



# Inside Test



## Exercise



The University of  
**Nottingham**

UNITED KINGDOM • CHINA • MALAYSIA

# Inside Test

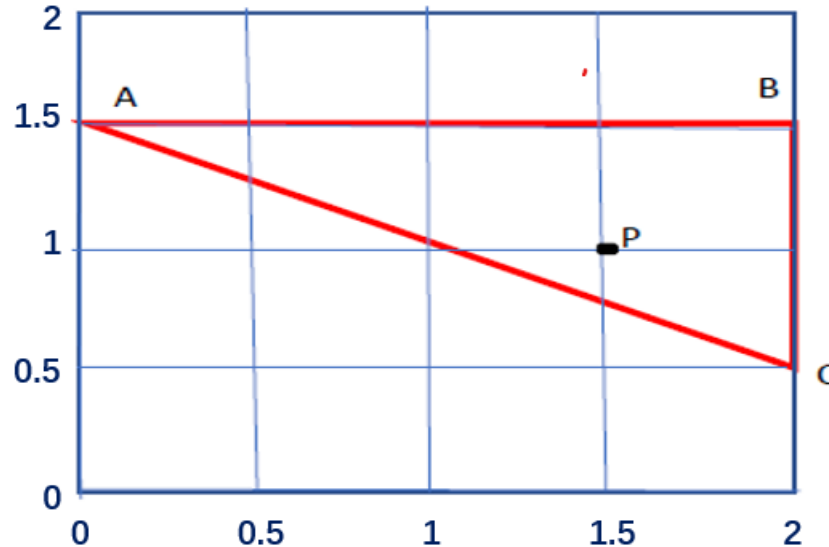
- ◆ Compute the Barycentric Coordinates  $\alpha$ ,  $\beta$  and  $\gamma$  for the following triangle ABC and point P. Is the point P inside the triangle?

A(0, 1.5)

B(2, 1.5)

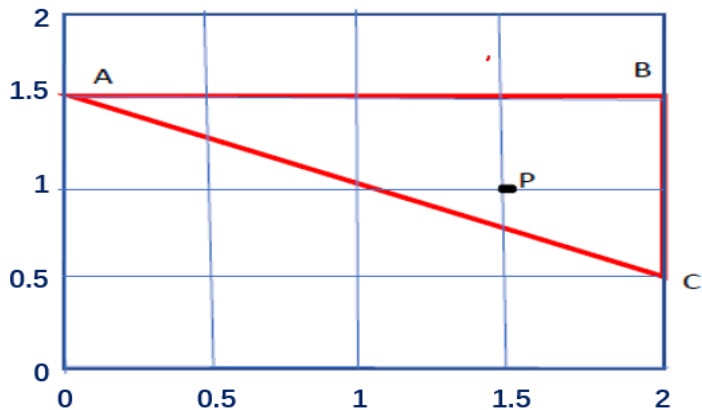
C(2, 0.5)

P(1.5, 1)



# Calculation of $\alpha$

$$\text{line}(F, S, T) = (S_y - F_y)T_x + (F_x - S_x)T_y + S_xF_y - F_xS_y$$



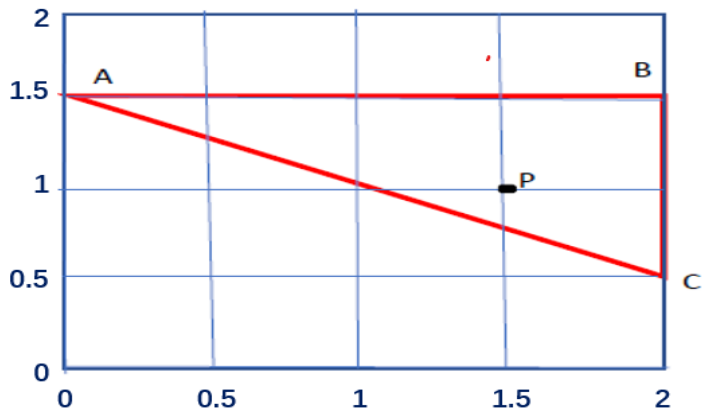
line (B,C,P) = ???

line (B,C,A) = ???

$$\alpha = \text{line (B,C,P)} / \text{line (B,C, A)}$$
$$= ???$$

# Calculation of $\beta$

$$\text{line}(F, S, T) = (S_y - F_y)T_x + (F_x - S_x)T_y + S_xF_y - F_xS_y$$



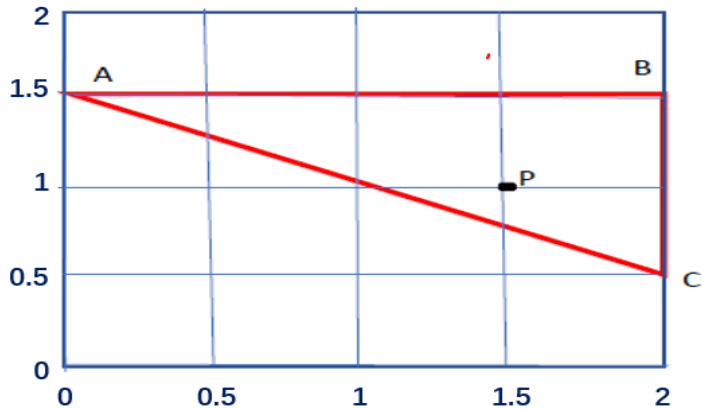
line (A,C,P) = ???

line (A,C,B) = ???

$$\beta = \text{line (A,C,P)} / \text{line (A,C, B)}$$
$$= ???$$

# Calculation of $\gamma$

$$\text{line}(F, S, T) = (S_y - F_y)T_x + (F_x - S_x)T_y + S_xF_y - F_xS_y$$



line (A,B,P) = ???

line (A,B,C) = ???

$$\gamma = \text{line (A,B,P)} / \text{line (A,B, C)} \\ = ???$$

# Is the point $P$ inside the triangle?



## Yes or No?

# References

- ◆ *COMP3069 Lecture notes: Chapter 2 - Rendering, Triangles, Rasterisation & Shading*
- ◆ *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL, Chapter 1*